

GeeksforGeeks

A computer science portal for geeks

GeeksQuiz

- [Home](#)
- [Algorithms](#)
- [DS](#)
- [GATE](#)
- [Interview Corner](#)
- [Q&A](#)
- [C](#)
- [C++](#)
- [Java](#)
- [Books](#)
- [Contribute](#)
- [Contests](#)
- [Jobs](#)

[Array](#)
[Bit Magic](#)
[C/C++](#)
[Articles](#)
[GFacts](#)
[Linked List](#)
[MCQ](#)
[Misc](#)
[Output](#)
[String](#)
[Tree](#)
[Graph](#)

Dynamic Programming | Set 3 (Longest Increasing Subsequence)

We have discussed Overlapping Subproblems and Optimal Substructure properties in [Set 1](#) and [Set 2](#) respectively.

Let us discuss Longest Increasing Subsequence (LIS) problem as an example problem that can be solved using Dynamic Programming.

The longest Increasing Subsequence (LIS) problem is to find the length of the longest subsequence of a given sequence such that all elements of the subsequence are sorted in increasing order. For example, length of LIS for { 10, 22, 9, 33, 21, 50, 41, 60, 80 } is 6 and LIS is {10, 22, 33, 50, 60, 80}.

Optimal Substructure:

Let $arr[0..n-1]$ be the input array and $L(i)$ be the length of the LIS till index i such that $arr[i]$ is part of LIS and $arr[i]$ is the last element in LIS, then $L(i)$ can be recursively written as.

$L(i) = \{ 1 + \text{Max} (L(j)) \}$ where $j < i$ and $arr[j] < arr[i]$ and if there is no such j then $L(i) = 1$

To get LIS of a given array, we need to return $\text{max}(L(i))$ where $0 < i < n$

So the LIS problem has optimal substructure property as the main problem can be solved using solutions to subproblems.

Overlapping Subproblems:

Following is simple recursive implementation of the LIS problem. The implementation simply follows the recursive structure mentioned above. The value of lis ending with every element is returned using `max_ending_here`. The overall lis is returned using pointer to a variable `max`.

```

/* A Naive recursive implementation of LIS problem */
#include<stdio.h>
#include<stdlib.h>

/* To make use of recursive calls, this function must return two things:
1) Length of LIS ending with element arr[n-1]. We use max_ending_here
   for this purpose
2) Overall maximum as the LIS may end with an element before arr[n-1]
   max_ref is used this purpose.
The value of LIS of full array of size n is stored in *max_ref which is our final result
*/
int _lis( int arr[], int n, int *max_ref)
{

```

```

/* Base case */
if(n == 1)
    return 1;

int res, max_ending_here = 1; // length of LIS ending with arr[n-1]

/* Recursively get all LIS ending with arr[0], arr[1] ... ar[n-2]. If
   arr[i-1] is smaller than arr[n-1], and max ending with arr[n-1] needs
   to be updated, then update it */
for(int i = 1; i < n; i++)
{
    res = _lis(arr, i, max_ref);
    if (arr[i-1] < arr[n-1] && res + 1 > max_ending_here)
        max_ending_here = res + 1;
}

// Compare max_ending_here with the overall max. And update the
// overall max if needed
if (*max_ref < max_ending_here)
    *max_ref = max_ending_here;

// Return length of LIS ending with arr[n-1]
return max_ending_here;
}

// The wrapper function for _lis()
int lis(int arr[], int n)
{
    // The max variable holds the result
    int max = 1;

    // The function _lis() stores its result in max
    _lis( arr, n, &max );

    // returns max
    return max;
}

/* Driver program to test above function */
int main()
{
    int arr[] = { 10, 22, 9, 33, 21, 50, 41, 60 };
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("Length of LIS is %d\n", lis( arr, n ));
    getchar();
    return 0;
}

```

Considering the above implementation, following is recursion tree for an array of size 4. lis(n) gives us the length of LIS for arr[].

```

          lis(4)
         /  |  \
      lis(3) lis(2) lis(1)
     /  \  /  \
  lis(2) lis(1) lis(1)
 /
lis(1)

```

We can see that there are many subproblems which are solved again and again. So this problem has Overlapping Substructure property and recomputation of same subproblems can be avoided by either using Memoization or Tabulation. Following is a tabulated implementation for the LIS problem.

```

/* Dynamic Programming implementation of LIS problem */
#include<stdio.h>
#include<stdlib.h>

/* lis() returns the length of the longest increasing subsequence in
   arr[] of size n */
int lis( int arr[], int n )
{
    int *lis, i, j, max = 0;
    lis = (int*) malloc ( sizeof( int ) * n );

    /* Initialize LIS values for all indexes */
    for ( i = 0; i < n; i++ )
        lis[i] = 1;

    /* Compute optimized LIS values in bottom up manner */
    for ( i = 1; i < n; i++ )
        for ( j = 0; j < i; j++ )
            if ( arr[i] > arr[j] && lis[i] < lis[j] + 1)

```

```

        lis[i] = lis[j] + 1;

    /* Pick maximum of all LIS values */
    for ( i = 0; i < n; i++ )
        if ( max < lis[i] )
            max = lis[i];

    /* Free memory to avoid memory leak */
    free( lis );

    return max;
}

/* Driver program to test above function */
int main()
{
    int arr[] = { 10, 22, 9, 33, 21, 50, 41, 60 };
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("Length of LIS is %d\n", lis( arr, n ) );

    getchar();
    return 0;
}

```

Note that the time complexity of the above Dynamic Programmig (DP) solution is $O(n^2)$ and there is a $O(n \log n)$ solution for the LIS problem (see [this](#)). We have not discussed the $n \log n$ solution here as the purpose of this post is to explain Dynamic Programmig with a simple example.

Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above.

Full access pass
to point of sale.

Related Topics:

- [Find Itinerary from a given list of tickets](#)
- [Load Balancing on Servers \(Random Algorithm\)](#)
- [Find number of Employees Under every Employee](#)
- [Expected Number of Trials until Success](#)
- [Linearity of Expectation](#)
- [Iterative Tower of Hanoi](#)
- [Count possible ways to construct buildings](#)
- [Build Lowest Number by Removing n digits from a given number](#)

Tags: [Dynamic Programming](#)

Like 27

Tweet

+1 5

Writing code in comment? Please use [ideone.com](#) and share the link here.

- *ayush*

In java (more efficient solution):

```

public static int[] solveUtil(int[] input, int n){

    if(n==1)

        return input;

    int max=0;

```

```

int index = 0;

int[] lis = new int[input.length];

lis[0] = input[0];

for (int i = 1; i < max){

    max=input[i];

    lis[++index]=input[i];

}

}

return lis;

}

```

o *kx*

This is wrong...

- *Guest*

Find the correct code here: <https://github.com/imperialguy/algorithms/blob/master/dynamic.py>

- *Ven Karri*

The code above is wrong!!

Find the correct code here:

<https://github.com/imperialguy/algorithms/blob/master/dynamic.py>

- *spIrs*

This is the version of code in java..

```

public class LongestIncreasingSubsequence{

    public static int arr[];

    public static void main(String[] args){

        arr=new int[]{10, 22, 9, 33, 21, 50, 41, 60, 80,100}; // Array length assume to be 10 ( It can be anything)

        System.out.println(find_list(arr,10));

    }

    public static int find_list(int a[],int len){

        int list[]=new int[10];

        for(int i=1;i<len;i++){

            for(int j=0;j<i;j++){

                if(a[j]<a[i] && list[i]<list[j]+1){

                    list[i]=list[j]+1;

                }

            }

        }

        return list[len-1];

    }

}

```

}

- *Aditya Pn*

Here is my solution with complexity of $O(n)$

<https://github.com/adityapn/ProgrammingInterview/blob/master/Dynamic%20Programming/LongestIncreasingSubSequence.py>

- *Bala*

Does your code work for 10,1,2,3,4,5?

I think ur code assumes LIS starts with first position of the sequence

- *Aditya Pn*

Thanks for finding out the bug 😊 , I corrected it to make work with all kinds of series . Check the same link again !

- *kx*

‘your code assumes LIS starts with first position of the sequence’ — That was not the issue with your code (If the code at above link is still good). I think your code doesn’t understand the ‘Longest Increasing Subsequence’.

For something like [11,12,13,14,15,1,2,3,16,17,18], longest common subsequence will be [11,12,13,14,15,16,17,18]. Above code will not find this.

- *Ven Karri*

LIS is not the same as longest contiguous increasing subsequence. Your program returns the longest contiguous increasing subsequence, but that wasn’t the question.

- ???

Non-recursive & recursive in same class, for comparing.

```
public class Solution {
    int[] max;
    int[] _max;
    public int LIS(int[] s) {
        max = new int[s.length];
        _max = new int[s.length];
        Arrays.fill(max, 1);
        Arrays.fill(_max, 1);
        for (int i = 1; i < s.length; i++) {
            for (int j = 0; j < i && max[i] < max[j] + 1)
                max[i] = max[j] + 1;
        }
    }
    //recursive method
    int _LIS(s, s.length);
    Arrays.sort(_max);
    //return max[s.length-1];
    return _max[s.length-1];
}
```

```

}

public int _LIS(int[] s, int where){
    if (where == 1)
        return 1;

    int length;

    for (int i = 1; i < _max[where-1] && s[i-1] < s[where-1])
        _max[where-1] = length + 1;
}

return _max[where-1];
}
}

```

- *leslie*

implemented _lis in java and for some reason fails when negative numbers appear in the array — any suggestions?

```

static public int lis (int arr[], int n, int maxRef) {
    if (n == 1) return 1;
    int res, max = 1;

    for (int i = 1; i < n; i++) {
        res = lis (arr, i, maxRef);
        if (arr[i - 1] > res) {
            max = res + 1;
        }
    }

    if (maxRef < max)
        maxRef = max;

    return max;
}

```

- *aman*

@Admin : Can we find the length of LIS using constant space only? Finding LIS wont be possible because output size isn't constant.

- *Pranali Yawalkar*

```

#include
#include
#include
#include

using namespace std;

int a[50];
std::pair LUT[10];

int function(int i);

int main()
{
    cout<<"enter a sequence"<<endl;
}

```

```
//int a[50];

for(int i=0;i>a[i];

LUT[0].first=a[0];

LUT[0].second=1;

function(1);

cout<LUT[i-1].first)

{

LUT[i].first=a[i];

LUT[i].second=LUT[i-1].second+1;

}

else

LUT[i]=LUT[i-1];

function(i+1);

}

}
```

- *Guest*

```
#include
#include
#include
#include
using namespace std;
int a[50];
std::pair LUT[10];
int function(int i);
int main()
{
cout<<"enter a sequence"<<endl;
for(int i=0;i>a[i];
LUT[0].first=a[0];
LUT[0].second=1;
function(1);
cout<LUT[i-1].first)
{
LUT[i].first=a[i];
LUT[i].second=LUT[i-1].second+1;
}
else
LUT[i]=LUT[i-1];
function(i+1);
}
}
```

- *Raghvendra Singh*

Hello Geeksforgeeks,

Please check your code for the input—

3,2,11,10,6,4,12,13,5,8

Although it is giving correct LIS which is 4 in this case but when i printed all the possible LIS of length 5 the result was wrong.

While i was checking the below two lines of code—
 if (arr[i] > arr[j] && lis[i] < lis[j] + 1)

```
lis[i] = lis[j] + 1;
```

i found that for value 8 in input sequence the LIS which i am getting is—
3,6,5,8
and this is incorrect because 5 cannot come after 6 in LIS.

Please clarify.

- *neelabhhsingh*

Dear Raghvendra,

I tested your test case. As given lis[], I print this array.

I am getting like this : list[]={1 1 2 2 2 2 3 4 3 4};

Length of LIS is 4

possible sub sequence are (3,2),(11,10,6,4) ,12,13 => length=4
(3,2), 4,(6,5),8=> length=4;
Not possible greater than 4.

- *pavansrinivas*

Following is $O(n^3)$ Soln that prints all the sequences...What modifications can be made to make it run in $O(n^2)$??

```
int LIS(int[] a){
    int fin_max = 1;
    for(int i=0;i<a.length;i++){
        int max = 1;
        for(int j=i+1;j<a.length;j++){
            int max_ele = a[i];
            String kk = a[i]+"-";
            for(int k = j;kmax_ele){
                max++;
                kk +=a[k]+"-";
                max_ele = a[k];
            }
        }
        System.out.println(kk+"----"+max);
        if(max>fin_max){
            fin_max = max;
        }
        max = 1;
    }
    return fin_max;
}
```

- *Saket Kumar*

// Java code time complexity $O(n^2)$ & space complexity $O(1)$

```
public static int LIS(int[] arr){
    int len=arr.length; int max_sub=0;

    for(int i=0;i<len;i++){
        int count=1; int value=arr[i];
        for(int j=i;j<len;j++){
            if(value<arr[j]){
                count++;
                value=arr[j];
            }
        }
        if(max_sub<count)
            max_sub=count;
    }
    return max_sub;
}
```

- *Chandan Mittal*

Can anyone please help me understanding what does lis[i] contains?

- *neelabhhsingh*

I am getting like this : list[]={1 1 2 2 2 2 3 4 3 4};

Length of LIS is 4

- *timus*

thanks for ths...

- *Subhransu Sahoo*

I do not think we need to loop through to find the max, the max is already there in lis[n-1]

```
/* Pick maximum of all LIS values */
for ( i = 0; i < n; i++ )
if ( max < lis[i] )
max = lis[i];
```

Just return lis[n-1]

- *sirui*

lis[n-1] may not be the max because lis[i] is “the length of the LIS till index i such that arr[i] is part of LIS “. Notice that arr[i] is part of LIS!

- *Sumit Monga*

No, the loop to find max is necessary because elements are in unsorted order and lis[i] is the longest increasing subsequence upto arr[i] with arr[i] included.
Hope its clear now.

- *anonymous*

suppose given array is
10,22,9,33,21,50,41,40,60,55
LCS 1,2, 1, 3, 2, 4, 4, 4, 5, 5

Explanation 10 there is no one smaller than 10 so value is LCS(10)=1 ,
LCS(22)= 2, LSC(9)=1 because we have to keep in mind that 9 is also included but there is no number less than 9,LSC(33)= 3 because 33 is bigger than all three number 10,22, 9 so take max LCS , which is LCS(22)=2,
LSC(33)=1+LSC(22) similar approach for all other ...

- *Shimpu*

```
for ( i = 0; i < n; i++ )
if ( max < lis[i] )
max = lis[i];
```

is necessary suppose the list is 10,22,9,33,21

lis values are 1,2,1,3,2

its lis[n-1] value is 2 ..which is incorrect...as it clearly shows ans is 3...so we hav to pick max frm lis...Hope you got the point...

- *ramneek garg*

```
#include
int max(int a,int b)
{
if(a>b)
return a;
else
return b;
}
int subseq(int i,int arr[])
{
static int longest[100];
int j,p=-1;
for(j=0;j {
if(arr[j]>arr[i])
```

```

{
    p=max(p,1);
}
else
{
    if(longest[j]==0)
    {
        p=max(subseq(j,arr),p);
    }
    else
    {
        p=max(longest[j],p);
    }
}
}
longest[i]=p+1;
return longest[i];
}
void main()
{
    int arr[]={10,22,9,33,21,50,41,60,80};
    printf("longest increasing subsequence is %d",subseq((sizeof(arr)/sizeof(int)-1),arr));
}

```

- *vinodhinic*

How to extend this logic to print all the LIS?

For eg:

for input {10,22,9,33,21,50,41,40,60,55}

LIS would be [1, 2, 1, 3, 2, 4, 4, 4, 5, 5]

And output should be:

```

10,22,33,40,55
10,22,33,41,60
10,22,33,50,60
10,22,33,41,55
10,22,33,50,55
10,22,33,40,60

```

I have tried using collections in JAVA

<http://ideone.com/H3zte5>

But I would appreciate if someone could post a neat solution

Thanks in advance.

- *Vinodhini*

Hi,

I need a solution that prints all the LIS of the given input. For Eg: for input {10,22,9,33,21,50,41,40,60,55} LIS array: [1, 2, 1, 3, 2, 4, 4, 4, 5, 5] The output should be:

```

10,22,33,40,55
10,22,33,41,60
10,22,33,50,60
10,22,33,41,55
10,22,33,50,55
10,22,33,40,60

```

I have tried in JAVA. (link: <http://ideone.com/H3zte5>) But to admit, it is pretty clumsy with all the collections and iterators. Could anyone provide solution that is simple? I am guessing we could use Graph and find the longest path. But I am nowhere near achieving the solution. Someone please help.

- *G4GFan*

```
/*
```

This prints length longest increasing subsequence (not necessarily contiguous)

```
{10,22,9,33,21,50,41,40,60,55} = 5
```

```
{10,22,33,50,60} = LIS
```

```

*/

#include

#include

#include

void printArray(int arr[], int len){
    int i = 0;
    while(i<len){
        printf("%dt", arr[i]);
        i++;
    }
    printf("n");
}

void printLis(int arr[], int lis[], int res[], int next, int index, int maxLength, int arrlen){
    int i,j,k;
    if(index == maxLength+1){
        printArray(res,index);
        return;
    }
    for(i=0;i0){
        k = 0;
        while(k lis[i]){
            break;
        }
        else {
            k++;
        }
    }
    if(k == i){
        res[next] = arr[i];
        //printf("%dn", res[next]);
        printLis(arr, lis, res, next+1, index+1, maxLength, arrlen);
    }
}

}

}

}

}

void getSubsequence(int arr[], int len){

```

```
int max,i,j,k,max_index;

int* lis = (int*)malloc(sizeof(int)*len);

max = arr[0];

lis[0] = 1;

i=1;

max_index = 0;

while(i= max){

lis[i] = lis[max_index] + 1;

max = arr[i];

max_index = i;

}

else{

int k = 0;

int min_diff = abs(arr[i]-arr[0]);

int index = 0;

while(k<i){

int diff = abs(arr[i]-arr[k]);

if(diff < min_diff){

min_diff = diff;

index = k;

}

k++;

}

if(arr[i] < arr[index]){

lis[i] = lis[index];

}

else{

lis[i] = lis[index] + 1;

}

}

i++;

}

printArray(lis, len);

int* res = (int*)malloc(sizeof(int) * lis[max_index]);

//printf("%dn", lis[max_index]);

printLis(arr, lis, res, 0, 1, lis[max_index], len);

}
```

```
int main(){
    int arr[] = {10,22,9,33,21,50,41,40,60,55};

    getSubsequence(arr, 10);

    int arr1[] = {9,33,21,50,40,60,55};

    getSubsequence(arr1, 7);

    return 0;
}
```

- *innosam*

<http://innosamcodes.wordpress.com/2013/07/06/longest-increasing-subsequence/>

Check out the blog, for a explanation with brevity.

I have also written code to print the LIS.

- *Karan*

Can somebody please explain why we need to check this [if (arr[i-1] max_ending_here)

max_ending_here = res + 1;

] condition?

- *sah_*

Yet another simple brute force $O(n^3)$ solution..

It considers all possible continuous sub-sequences...

```
#include
```

```
using namespace std;
int main()
{
    int arr[]={ 10, 22, 9, 33, 21, 50, 41, 60, 80 } ;
    int big=0;
    int curr=0;
    int n=9;
    for(int i=0;i {
        for(int j=i;j {
            curr=1;
            for(int k=i;k {
                if(arr[k] > curr;
            }
            if(big < curr)
            {
                cout<<"\nLIS is of length : "<< curr;
                return 0;
            }
        }
    }
}
```

- *Nikin Kumar Jain*

Method to Print Longest Increasing Subsequence

```
// LongestIncreasingSequence.cpp : Defines the entry point for the console application.
//
```

```
#include "stdafx.h"
#include <iostream>
```

```
using namespace std;
```

```
void printLIS(int arr[], int lis[], int max, int n)
{
    if(n < 0)
        return;
    if(lis[n] == max)
    {
        printLIS(arr, lis, max-1, n-1);
        cout<<arr[n]<<" ";
    }
    else
```

```

    {
        printLIS(arr, lis, max, n-1);
    }
}

int lis(int arr[], int n)
{
    int count = 0;
    int *lis = (int *)malloc(sizeof(int) * n);
    int i, j, index;

    for(int i=0;i<n;i++)
        lis[i] = 1;

    for(int i=1;i<n;i++)
        for(int j = 0; j<i;j++)
        {
            if(arr[i] > arr[j] && lis[i] < lis[j] + 1)
            {
                lis[i] = lis[j] + 1;
                index = i;
            }
        }
    int max = lis[index];
    cout<<endl;

    printLIS(arr, lis, max, n-1);

    free(lis);
    return max;
}

int _tmain(int argc, _TCHAR* argv[])
{
    int arr[] = {10,22,9,33,21,50,41,60};

    cout<<endl<<"Longest Increasing Sequence: "<<lis(arr, sizeof(arr)/sizeof(arr[0]));
    getchar();
    return 0;
}

```

o *shubham*

there is something wrong in printing values as it is printing wrong for test case

{ 10, 22, 9, 33, 21, 50, 41, 60, 80 }

it should print { 10,22,33, 50, 60, 80 }

but it is giving { 10,22,33 ,41 ,60 ,80 }

/* Paste your code here (You may delete these lines if not writing code) */

■ *shubham*

//it should be like this

```

#include
#include
using namespace std;
void printLIS(int arr[], int lis[], int max, int n)
{
    if(n 0;i--)
    {
        if(lis[n]==max && lis[n-1]==max && arr[n]<arr[n-1])
            n=n-1;
        else
            break;
    }
    printLIS(arr, lis, max-1, n-1);
    cout<<arr[n]<<" ";
}
else
{
    printLIS(arr, lis, max, n-1);
}
}

```

```

void lis_length(int arr[],int m)
{
    int lis[m+1];
    int value[m+1];
    fill(&lis[0],&lis[m],1);
    for(int i=1;i<m;i++)
        for(int j=0;j<i;j++)
            if(arr[j]<arr[i])
                lis[i]=lis[j]+1;

    int max=*max_element(lis,lis+m);
    cout<<max<<endl;
    while(t--)
    {
        int n;
        cin>>n;
        int a[n];
        for(int i=0;i<n;i++)
            a[i]=arr[i];

        lis_length(a,n);
    }
}

```

■ shubham

```

#include
#include
using namespace std;
void printLIS(int arr[], int lis[], int max, int n)
{
    if(n 0;i--)
    {
        if(lis[n]==max && lis[n-1]==max && arr[n]<arr[n-1])
            n=n-1;
        else
            break;
    }
    printLIS(arr, lis, max-1, n-1);
    cout<<arr[n]<<" ";
}
else
{
    printLIS(arr, lis, max, n-1);
}
}

```

```

void lis_length(int arr[],int m)
{
    int lis[m+1];
    int value[m+1];
    fill(&lis[0],&lis[m],1);
    for(int i=1;i<m;i++)
        for(int j=0;j<i;j++)
            if(arr[j]<arr[i])
                lis[i]=lis[j]+1;

    int max=*max_element(lis,lis+m);
    cout<<max<<endl;
    while(t--)
    {
        int n;
        cin>>n;
        int a[n];
        for(int i=0;i<n;i++)
            a[i]=arr[i];

        lis_length(a,n);
    }
}

```

■ aygul

Actually it not wrong. It is one of the LISs and there might be more than one.

If you extend the code by storing the max end points, with the help of max length you can print all LISs.

o *mohitk*

Hi,
I believe the above also doesn't work for the below input:
{10, 11, 12, 9, 8, 7, 5, 6 }

It would always select the last altered LIS value as max, which is not the case in above.
The algo returns 2 whereas correct should be 3.

```
/* Paste your code here (You may delete these lines if not writing code) */
```

■ *Rohit*

@mohitk : yea.. it would not print proper values whenever the last element in sequence is not maximum, so it just need to copy its max value calculation from geeksforgeeks solution(not to mention, with proper curly braces) instead of making "index = i" and it should work fine.

```
/* Pick maximum of all LIS values */
for ( i = 0; i < n; i++ ){
    if ( max < lis[i] )
        max = lis[i];
}
```

```
/* Paste your code here (You may delete these lines if not writing code) */
```

• *mrn*

```
/* Paste your code here (You may delete these lines if not writing code) */
for(int i=0;i<n;i++)
    for(int j=i+1;j<n;j++)
    {
        if(a[i] < a[j])
        {
            b[j]=max(b[j],b[i]+1);
            if(maxim<b[j])
                maxim=b[j];
            prv[j]=i;
        }
    }
```

• *jatinjindalj*

Hello Geeksforgeeks,

I tried to understand and code the solution in Java.
The below is the code and it looks a little simpler than what you have given.
Could not understand the purpose of max_ref.
Can you please check.

Regards

```
public class lis
{
    static int lis_(int[] in,int last)
    {
        if(last==1)
            return 1;
        int result,maxTillNow=1;
        for(int i=1;i<last;i++)
        {
            result=lis_(in,i);
            if(in[i-1]<in[last-1] && result+1>maxTillNow)
                maxTillNow=result+1;
        }
        return maxTillNow;
    }

    public static void main(String args[])
    {
    }
```



```
{
int[] in={0,8,4,12,2,10,6,14,1,9,5,13,3,11,7,15};
System.out.println("lis:"+lis_(in,in.length));
}
}
```

- *maverick*

@GeeksforGeeks There is a minute bug in the recursive code above in the line

```
if (arr[i-1] max_ending_here)
```

which should actually be

```
if (arr[i-1] max_ending_here)
```

- *bharath Jhadey*

/* Paste your code here (You may delete these lines if not writing code) */

```
public class LCS {
public LCS() {
super();
}
public static int LCS(int [] A, int current, int k){
if(current>=A.length)
return 0;
if(A[current]>k)
return max(1+LCS(A,current+1,A[current]),LCS(A,current+1,k));
else return LCS(A,current+1,A[current]);
}
public static int max(int a,int b){
return a>b?a:b;
}
public static void main(String [] args){
int [] A={10,22,9,33,21,50,41,60,80};
System.out.println( LCS(A,0,0));
}
}
```

- *Saurabh Jain*

/* Paste your code here (You may delete these lines if not writing code) */

```
/**
 *
 * @author saurabh
 */
public class LongestIncreasingSubsequence
{
    int a[];
    public LongestIncreasingSubsequence()
    {
        Scanner sc = new Scanner(System.in);
        int size = sc.nextInt();
        a = new int[size];
        for(int i=0; i<size; i++)
        {
            a[i] = sc.nextInt();
        }
        System.out.println("Initial Array : ");
        this.printArray();
        System.out.println("\nLongest Increasing SubSequence is : ");
        this.printLongestSubSequence();
    }

    private void printArray()
    {
        for(int i=0; i<a.length; i++)
            System.out.print(a[i]+" ");
    }

    private void printLongestSubSequence()
    {
        int max_len=1,len_so_far,init=0,1;
```

```

for(int i=0; i<a.length; i++)
{
    len_so_far=1;
    l = a[i];
    for(int j=i+1; j<a.length; j++)
    {
        if(a[j]>l)
        {
            len_so_far++;
            l = a[j];
        }
    }
    if(max_len<len_so_far)
    {
        max_len = len_so_far;
        init = i;
    }
}
int count=1;
l = a[init];
System.out.print(a[init]+" ");
for(int i=init+1; i<a.length; i++)
    if(a[i]>l)
    {
        System.out.print(a[i]+" ");
        l = a[i];
        count++;
    }
System.out.println();
System.out.println("Length of Longest Increasing SubSequence is : "+count);
}

public static void main(String[] args)
{
    LongestIncreasingSubsequence lis = new LongestIncreasingSubsequence();
}
}

```

◦ *Saurabh Jain*

Time Complexity of this code is $O(n*n)$ or $O(n^2)$...correct me if anything is wrong.....

◦ *Saurabh Jain*

I forgot to write.... import java.util.Scanner....so don't forget to use it...

• *swathi*

Can you please explain how to print the sequence of the LIS. I am facing the difficulty when we have multiple sets of LIS. Thanks..

• *Spock*

Why is there a need to check $lis[i] < lis[j] + 1$??
I guess $lis[i]$ will always be greater than $lis[j] + 1$.

◦ *Shrey*

@Spock : Consider for $i=3$; you have the arr array's elements as {10,22,9,33}. Now for calculating lis array we compare 33 with 10 its ok then we compare 33 with 22 its again ok and at last 33 with 9, its ok but the given subsequence is not a longest increasing subsequence as $9 < 22$. hence this check is required.

Correct me if i am wrong.

• *supercooluku*

What is the time complexity of the naive recursive solution of LIS and also tell me how to find the time complexity of such a recursive code...

◦ *kartik*

The time complexity of the Naive Recursive solution is $O(2^{(n-1)})$ (2 raise to the power $n-1$).

You can get the time complexity by taking some sample values. Like $T(3) = 4$, $T(4) = 8$, $T(5) = 16$.

You can also solve it by solving the following recurrence.

$T(n) = T(n-1) + T(n-2) + .. T(1)$

and $T(1)$ is constant

- *krishna*

The problem “sum of the sub sequence”, cannot have $O(n \log n)$.

Reason: if we encounter an element which is less than all the max heads we need to move the array completely to right by one and add one element there, is there any other logic that avoids this condition? i believe not.

- *GeeksforGeeks*

@venki & @sparco:

The recursive function “_lis(arr, n)” must return the length of LIS ending arr[n-1] so that the value can be used by caller in recursion. Also, we need length of LIS in overall array. The earlier implementation had some issues as it was returning only one value. Thanks for pointing out the issues. We have updated the naive recursive implementation now.

- *Venki*

@Sandeep, In the recursive program, the logic behind the comparison “if (arr[i-1] < arr[n-1])” is not clear.

Consider we are at an arbitrary position ‘i’, and want to find the length of LIS at i-th location. If we know the maximum length till (i-1)th element, and if ($A[i] > A[i-1]$) we simply extend the LIS length by one. Otherwise (i.e. ($A[i] \leq A[i-1]$)), the maximum length of LIS at i-th location is same as (i-1)th location.

Putting in code, I have the following piece of code,

```
int LIS(int A[], int l, int r)
{
    if( (r - l + 1) == 1 )
        return 1;

    int curLen;
    int maxLen = 1;
    for( int i = l; i <= r; i++ )
    {
        curLen = LIS(A, l, i-1);

        if( A[i] > A[i-1] )
            curLen++;

        MaxSampleAndHold(maxLen, curLen);
    }

    return maxLen;
}
```

It is slightly different from the recursive logic given in the post. Is there any counter example for the above code?

- *sparco*

@geeksforgeeks

In recursion logic ,

I dont get why (how the below code1 works exactly as code2)

Code1 – as written in the page

```
if (arr[i-1] < arr[n-1])
    res++;
```

is used

Code2

```
if (arr[i-1] < arr[i])
    res++;
```

is not used according to notes

To make use of recursive calls, this function must return two things:

1) Length of LIS

2) Index of the last element in LIS (This is necessary as we need to compare the last element for new LIS)

- *kartikaditya*

```
public class LongestIncreasingSubsequence {
    private static int binarySearch(int a[], int x, int start, int end) {
        int limit = end;
        int mid = 0;
        while (start < end) {
            mid = (start + end) >> 1;
            if (a[mid] <= x) {
                start = mid + 1;
            } else {
                end = mid - 1;
            }
        }
        if (a[start] <= x) {
            return start;
        }
        return start - 1;
    }

    public static void getLongestIncreasingSubsequence(int a[]) {
        int dp[] = new int[a.length + 1];
        int memo[] = new int[a.length + 1];

        dp[0] = 1;
        memo[1] = a[0];
        int maxSize = 1;
        for (int i = 1; i < a.length; ++i) {
            if (a[i] < memo[1]) {
                memo[1] = a[i];
                dp[i] = 1;
            } else if (a[i] >= memo[maxSize]) {
                ++maxSize;
                memo[maxSize] = a[i];
                dp[i] = maxSize;
            } else {
                int k = binarySearch(memo, a[i], 1, maxSize);
                memo[k + 1] = a[i];
                dp[i] = k + 1;
            }
        }
        System.out.print(maxSize + " :: ");
        for (int i = 1; i <= maxSize; ++i) {
            System.out.print(memo[i] + " ");
        }
        System.out.println();
    }

    public static void main(String args[]) {
        getLongestIncreasingSubsequence(new int[] {10, 22, 9, 33, 21, 50, 41, 60});
        getLongestIncreasingSubsequence(new int[] {10, 22, 9, 33, 21, 50, 41, 60, 80});
        getLongestIncreasingSubsequence(new int[] {1, 1, 1, 1, 0, 0, 0, 0, 0});
        getLongestIncreasingSubsequence(new int[] {0, 0, 0, 0, 0, 1, 1, 1, 1});
    }
}
```

- *Devanshkaushik*

hey, @kartikaditya, i tried ur code in netbeans, it gave the following output, while 33 should not be there i gues....

```
run:
5 :: 9 21 33 41 60
6 :: 9 21 33 41 60 80
5 :: 0 0 0 0 0
9 :: 0 0 0 0 0 1 1 1 1
```

and more in case 2, there is another sequence 10,22,33,50,60,80 which also has length 6, so why only this is printed?
plzz clarify...

◦ *Devanshkaushik*

hey, @kartikaditya, i tried ur code in netbeans, it gave the following output, while 21 should not be there i guess....

```
run:
5 :: 9 21 33 41 60
6 :: 9 21 33 41 60 80
5 :: 0 0 0 0 0
9 :: 0 0 0 0 0 1 1 1 1
```

and more in case 2, there is another sequence 10,22,33,50,60,80 which also has length 6, so why only this is printed?
plzz clarify...

• *Aashish Barnwal*

One solution is the modified version of longest increasing sub-sequence.
Here, take the first string as the sequence given,
second string is the sorted order of the first string.
Now apply the LCS algorithm.

```
/*
#include <stdio.h>
#define SIZE1 9
#define SIZE2 9

void lcs(char *s1, char *s2)
{
    int l[SIZE1+1][SIZE2+1], i, j;
    char path[SIZE1+1][SIZE2+1];
    for(i=0; i<=SIZE1; i++)
        l[i][0]=0;
    for(i=0; i<=SIZE2; i++)
        l[0][i]=0;

    for(i=1; i<=SIZE1; i++)
        for(j=1; j<=SIZE2; j++)
        {
            if(s1[i-1]==s2[j-1])
            {
                l[i][j]=l[i-1][j-1] + 1;
                path[i][j]='D';
            }
            else if(l[i][j-1] > l[i-1][j])
            {
                l[i][j]=l[i][j-1];
                path[i][j]='L';
            }
            else
            {
                l[i][j]=l[i-1][j];
                path[i][j]='T';
            }
        }
    i=SIZE1, j=SIZE2;
    while(i>0 && j>0)
    {
        if(path[i][j]=='T')
            i--;
        else if(path[i][j]=='L')
            j--;
        else
        {
            printf("%c ", s1[i-1]);
            i--, j--;
        }
    }
    printf("\n");
    /*for(i=0; i<=SIZE1; i++)
    {
        for(j=0; j<=SIZE2; j++)
            printf("%d ", l[i][j]);
        printf("\n");
    }
    */
}
```

```

int main()
{
    char s1[SIZE1+1],s2[SIZE2+1];
    printf("Enter String1 n String2 ");
    scanf("%s%s",s1,s2);
    lcs(s1,s2);
    return 0;
}
*/

```

- *Venkatesh*

Can some one help me how to print the increasing sequence?

- *outkast*

Recursive Solution fails for input

int arr[] = {6,7,8,9,1};

- *outkast*

The Length of LIS is 4(6,7,8,9) but the function returns 1.

- *GeeksforGeeks*

@outkast: Thanks for pointing this out. There were issues in the previous code. We have updated the code.

- *Abhinav Kumar*

The code given using simple recursive is not consistent as it dose not follow what the question asks.

To try just take input array : {10,22,4,5,6,9,33,50,60,80}

- <http://geeksforgeeks.org/Sandeep>

@Abhinav Kumar: Thanks for pointing this out. The code doesn't seem to work for this example. We will look into this and fix the code.

- <http://geeksforgeeks.org/Sandeep>

@Abhinav Kumar: I have updated the code and it now works fine for the example given by you.

```

/* A Naive recursive implementation of LIS problem */
#include<stdio.h>
#include<stdlib.h>

int lis( int arr[], int n )
{
    int max = 1;
    int ls;

    /* Base case */
    if(n == 1)
        return max;

    /* Recursively get LIS for all smaller elements on left side */
    for(int i = 0; i < n - 1; i++)
    {
        if (arr[i] < arr[n-1])
        {
            ls = lis(arr, i+1) + 1;

            /* Check if max needs to be updated */
            if(ls > max)
                max = ls;
        }
    }
    return max;
}

/* Driver program to test above function */
int main()
{
    int arr[] = {10,22,4,5,6,9,33,50,60,80};
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("Length of LIS is %d\n", lis( arr, n ) );
}

```

```

getchar();
return 0;
}

```

■ Yueming

Try this {10,22,4,5,6,9,33,50,60,80,0}, it return 1!

/* Paste your code here (You may delete these lines if not writing code) */

■ Sandeep

The code was further updated due to one more issue pointed by a user. The final code given in post works for your example.

```

/* A Naive recursive implementation of LIS problem */
#include<stdio.h>
#include<stdlib.h>

/* To make use of recursive calls, this function must return two things:
1) Length of LIS
2) Index of the last element in LIS (This is necessary as we need to compare
the last element for new LIS)
*/

int lis( int arr[], int n )
{
    /* Base case */
    if(n == 1)
        return 1;

    int res, max = 1;

    /* Recursively get LIS for all smaller elements on left side */
    for(int i = 1; i < n; i++)
    {
        res = lis(arr, i);
        if (arr[i-1] < arr[n-1])
            res++;

        /* Check if max needs to be updated */
        if(res > max)
            max = res;
    }
    return max;
}

/* Driver program to test above function */
int main()
{
    int arr[] = {10,22,4,5,6,9,33,50,60,80,0};
    int n = sizeof(arr)/sizeof(arr[0]);
    int max = lis( arr, n );
    printf("Length of LIS is %d\n", max);

    getchar();
    return 0;
}

```

• KK123

This is the link to O(nlogk) implementation:

http://www.algorithmist.com/index.php/Longest_Increasing_Subsequence.cpp

• pucitian

i am soin my term paper on dp using lcs as an example.....i found this topic very interesting.....

• vignesh

In the above paragraph explaining optimal substructure i find something missing.

Let [0..n-1] be the input array and L(i) be the length of the LIS till index i including ith index then L(i) can be recursively written as.

$LIS(i) = \{ 1 + \text{Max} (LIS(j)) \}$ where $j < i$ and $arr[j] < arr[i]$ and if there is no such j then "**LIS(i) = 1**"

It should be "**Lis(i) = Lis(i-1)**".

For instance if 10, 8, 6, 11, 5 is the input array.

Lis(0) = 1 // 10 alone

Lis(1) = 1 // 8 or 10 alone

Lis(2) = 1 // 6 or 8 or 10 alone

Lis(3) = 2 // {10,11 } or {8,11} or {6,11}

Lis(4) = 2 // {10,11 } or {8,11} or {6,11} though 5 is not included. but according to the recurrence relation mentioned in the above paragraph Lis(4) will be one which is erroneous against the basic assumption that Lis(i) represents longest increasing subsequence including i.

- <http://geeksforgeeks.org/Sandeep>

@vignesh: Thanks for pointing this out. There was a typo in explanation, I have corrected it so that it matches with the implementation. Let me know if it looks fine now.

■ vignesh

@sandeep :- my understanding about the recurrence relation was wrong. Lis(i) represents longest subsequence ending in i (i mistook it as including i). so recurrence relation is right. After populating the Lis array. One should find the maximum in Lis[] to get LIS. Well in nlogn approach they maintain a separate array to keep track of individual elements as well.

■ SDK

I still don't get geeksforgeeks explanation that why LIS[i] = 1 ... it should be LIS[i] = LIS[i-1] if the conditions are not met....

I think the following code works plz check.

```
#include<stdio.h>
#include<stdlib.h>
int maxof(int *a,int n){
    int i=0;
    int max=a[0];
    for(i=0;i<n;i++){
        if(a[i]>max) max=a[i];
    }
    return i;
}
int lis(int arr[],int n){

    int i,j,index;
    int*ls=(int*) calloc(sizeof(int),n);

    ls[0]=1; //if it contains only one element;

    for(i=1;i<n;i++){
        for(j=0;j<i;j++){

            index = maxof(ls,i-1);
            if(arr[i]>arr[index])
                ls[i]=1 + ls[index];
            else
                ls[i] = ls[i-1];
        }
    }

    return ls[n-1];
}

int main()
{
    int arr[] = {10,20,11,12,22,44,62,8,82};
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("Length of LIS is %d\n", lis( arr, n ) );

    getchar();
    return 0;
}
```

plz explain why they have taken LIS[i]=1

■ GeeksforGeeks

@SDK: Every element of input array is LIS of length 1. That is why we have initial values in LIS as

1.

- *martin*

@GeeksforGeeks Note that the time complexity of the above Dynamic Programmig (DP) solution is $O(n^2)$ and there is a $O(n \log n)$ solution for the LIS problem (see this). We have not discussed the $n \log n$ solution here as the purpose of this post is to explain Dynamic Programming with a simple example.

so please don't worry about complexity , please try to explain your $O(n \log n)$ solution , i wants to learn from you geeksforgeeks , please help us 😊 & keep it up 😊

- http://geeksforgeeks.org/?page_id=2 Venki

@Martin, Thanks for your comments, we will soon have the $n \log n$ solution of LIS as a post under special category.

- *reema*

@sandeep its will just return the length of LIS not LIs itself , can you post the solution for that itself or please tell me wht modification we need to do for that to print LIS from above array ?

- *satya*

@GeeksforGeeks..Keep Posting FAQ DP Asked problems ..keep it up..

- *Vinay*

plz how are “Longest Increasing Subsequence problem” and “Maximum Sorted(ascending) Subsequence problem” diffrent?

- *giri*

Good job. I find following practice problems also very useful.

<http://people.csail.mit.edu/bdean/6.046/dp/>

- *tk*

Can someone explain the $n \log n$ approach for LIS?

- *GeeksforGeeks*

@tk: We will soon publish a separate post for $n \log n$ solution of LIS problem.

- *Anand*

<http://anandtechblog.blogspot.com/2010/06/longest-common-subsequence.html>

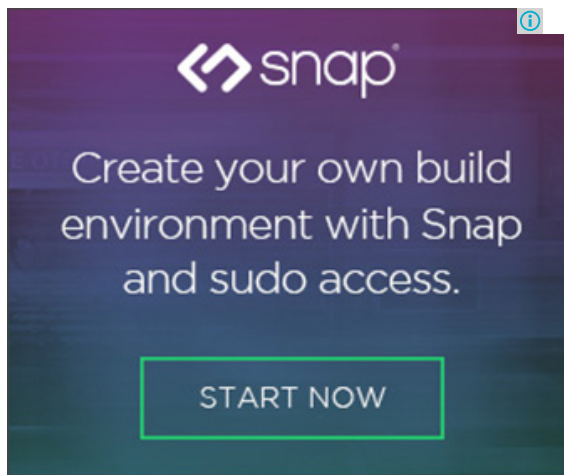
- *GeeksforGeeks*

@venki: Thanks for pointing this out. I have corrected the typo.

- *reema*

@GeeksForGeeks its will just return the length of LIS not LIs itself , can you post the solution for that itself or please tell me wht modification we need to do for that to print LIS from above array ?

-
-



-
- - [Interview Experiences](#)
 - [Advanced Data Structures](#)
 - [Dynamic Programming](#)
 - [Greedy Algorithms](#)
 - [Backtracking](#)
 - [Pattern Searching](#)
 - [Divide & Conquer](#)
 - [Mathematical Algorithms](#)
 - [Recursion](#)
 - [Geometric Algorithms](#)



-
- **Popular Posts**

- [All permutations of a given string](#)
- [Memory Layout of C Programs](#)
- [Understanding “extern” keyword in C](#)
- [Median of two sorted arrays](#)
- [Tree traversal without recursion and without stack!](#)
- [Structure Member Alignment, Padding and Data Packing](#)

- [Intersection point of two Linked Lists](#)
- [Lowest Common Ancestor in a BST](#)
- [Check if a binary tree is BST or not](#)
- [Sorted Linked List to Balanced BST](#)

Follow @GeeksforGeeks

Recent Comments

- [Vaibhav](#)

<http://stackoverflow.com/quest...>

[Payu Interview \(On-Campus\)](#) · 47 minutes ago

- [varun](#)

Traversal using arbit pointer always forms...

[Point to next higher value node in a linked list with an arbitrary pointer](#) · 48 minutes ago

- [Tanish Bansal](#)

int com(node *root){ if(!root) return 0;...

[Check whether a given Binary Tree is Complete or not](#) · 57 minutes ago

- [dextrous](#)

According to me there's no point in doing that...

[Write a C program to Delete a Tree.](#) · 1 hour ago

- [guest](#)

int zero=1; int one=1; for(int i=2;i<n;i++) {...="">

[Count number of binary strings without consecutive 1's](#) · 2 hours ago

- [Garima Dhanania](#)

what if we write root=NULL after free(n) inside...

[Write a C program to Delete a Tree.](#) · 2 hours ago

Ads by Google

► [Geeks for Geeks](#)

► [Array](#)

► [Sequence](#)

Ads by Google

► [Algorithm](#)

► [C++ Code](#)

► [Int](#)

Ads by Google

► [C++ Program](#)

► [Python C++](#)

► [Recursion](#)

@geeksforgeeks, [Some rights reserved](#) ____ [Contact Us!](#) ____ [Abut Us!](#)

Powered by [WordPress](#) & [MooTools](#), customized by geeksforgeeks